Clase05_SVM_JorgeLuisGuzman

May 11, 2026

0.0.1 SVMs y optimización de hiperparámetros

En este notebook trabajaremos con un dataset simulado de eventos del experimento ATLAS. Cada fila representa un evento de colisión protón–protón, y cada columna corresponde a una variable reconstruida a partir de los objetos físicos detectados en el evento: jets, leptones, energía faltante, etc.

El objetivo es distinguir entre dos tipos de eventos:

- **`ttbar`**: producción de un par top–antitop. Es un proceso mucho más frecuente.
- **`4top`**: producción de cuatro quarks top. Es un proceso mucho más raro y corresponde a nuestra clase positiva.

Basado en Machine learning for physics and Astronomy, Viviana Acquaviva (2023)

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import rc
from sklearn.svm import SVC, LinearSVC # Nuevo algoritmo!
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_predict, cross_validate
from sklearn.model_selection import KFold, StratifiedKFold
from sklearn import metrics
from sklearn.model_selection import GridSearchCV # Nuevo! Para explorar los␣
↪hiperparámetros del modelo
```

[2]:
```
pd.set_option('display.max_columns', 500)
pd.set_option('display.max_rows', 500)
pd.set_option('display.max_colwidth', 100)
rc('text', usetex=False)
```

Leemos las características y las etiquetas

[3]:
```
features = pd.read_csv('ParticleID_features.csv', index_col='ID')
```

[4]:
```
features.head(10)
```

[4]:
```
           MET    METphi Type_1        P1        P2        P3        P4 Type_2  \
ID
0     62803.50 -1.810010      j  137571.0  128444.0 -0.345744 -0.307112      j
```

```
1    57594.20 -0.509253      j  161529.0   80458.3 -1.318010  1.402050      j
2    82313.30  1.686840      b  167130.0  113078.0  0.937258 -2.068680      j
3    30610.80  2.617120      j  112267.0   61383.9 -1.211050 -1.457800      b
4    45153.10 -2.241350      j  178174.0  100164.0  1.166880 -0.018721      j
5    43803.10  0.572184      b  146274.0  111016.0 -0.776311  1.083980      j
6    28100.40 -0.593429      j  458244.0  391557.0 -0.546723  0.013275      b
7   166458.00  0.473635      j  848058.0  273398.0 -1.794510  2.763560      j
8     4664.13 -1.828120      j  477219.0  331550.0 -0.899484  0.698675      b
9    17426.90  1.363170      j   74241.5   71823.2 -0.228244 -1.755950      j

            P5        P6        P7        P8 Type_3        P9       P10  \
ID
0    174209.0  127932.0  0.826569  2.332000      b   86788.9   84554.9
1    291490.0   68462.9 -2.126740 -2.582310     e-   44270.1   35139.6
2    102423.0   54922.3  1.226850  0.646589      j   60768.9   36244.3
3     40647.8   39472.0 -0.024646 -2.222800      j  201589.0   32978.6
4     92351.3   69762.1  0.774114  2.568740      j   61625.2   50086.7
5   1036150.0  109475.0 -2.937790 -1.518380      j  259026.0   71613.8
6    425427.0  328566.0 -0.728581 -2.411570      j  126694.0  116820.0
7    267149.0  242308.0 -0.425923 -1.534860      b  461064.0  159944.0
8    696365.0  246682.0 -1.697140 -2.285070      j  987005.0   62516.5
9     60026.7   54692.0  0.414105  1.453790      j  217944.0   53033.2

         P11       P12 Type_4       P13       P14       P15       P16 Type_5  \
ID
0  -0.180795  2.187970      j  140289.0   76955.8 -1.199330 -1.302800     m+
1  -0.706120 -0.371392     e+   72883.9   26902.2 -1.653860 -3.129630    NaN
2   1.102890 -1.434480      j   77714.0   27801.5  1.684610  1.389690      j
3  -2.496040  1.137810      j   90096.7   26964.5  1.871320  0.817631      j
4   0.652572 -3.012800      j  104193.0   31151.0  1.876410  0.865381      j
5  -1.955480 -1.726540      j  137198.0   59925.9 -1.469500  2.189410      j
6  -0.350992  2.208790      j  112382.0  111318.0  0.073453  3.030960      j
7  -1.719880  1.094360      j   75909.0   73858.8 -0.235076 -0.330519      b
8  -3.451230 -2.207720      j   51561.9   36406.3 -0.881624 -2.923760      j
9  -2.091050 -1.113380      j   60286.5   41261.5 -0.922507 -2.192660      j

          P17       P18       P19       P20 Type_6       P21       P22  \
ID
0     85230.6  70102.4 -0.645689 -1.659540    NaN       NaN       NaN
1         NaN       NaN       NaN       NaN    NaN       NaN       NaN
2     26840.0  24469.3 -0.388937 -1.647260    NaN       NaN       NaN
3     28235.4  25887.9 -0.411528  2.024290    NaN       NaN       NaN
4    746585.0  26219.3  4.041820 -0.874169    NaN       NaN       NaN
5     95419.6  55847.5  1.123310  0.846395      j  451389.0  53178.9
6   3039490.0  65143.9  4.535880  1.516880      j   86235.2  65037.1
7    201351.0  53338.2 -2.002180 -2.376840      b   38794.0  37225.5
8     85262.1  35203.6 -1.527390  2.071480      j  202166.0  26361.8
```

```
9     56587.7  36051.0 -1.013930  1.468390      j   36707.5  28128.1

         P23      P24 Type_7       P25      P26       P27       P28 Type_8  \
ID
0        NaN      NaN    NaN       NaN      NaN       NaN       NaN    NaN
1        NaN      NaN    NaN       NaN      NaN       NaN       NaN    NaN
2        NaN      NaN    NaN       NaN      NaN       NaN       NaN    NaN
3        NaN      NaN    NaN       NaN      NaN       NaN       NaN    NaN
4        NaN      NaN    NaN       NaN      NaN       NaN       NaN    NaN
5  -2.827840 -2.47964      j   51436.4  50713.2  0.056029  2.258540      j
6   0.780704  1.50869      b   62675.6  61535.7  0.099510  0.466516      j
7   0.187429 -2.78635      b   40323.4  36915.1  0.411490  1.844490      j
8  -2.725680 -2.21350      j   56071.6  25934.6 -1.403180  1.130990    NaN
9  -0.750243  2.62670      j  691646.0  25461.4  3.994660  0.641807    NaN

         P29      P30      P31       P32 Type_9       P33      P34      P35  \
ID
0        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
1        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
2        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
3        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
4        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
5   152293.0  48842.5 -1.80056 -0.704609    NaN       NaN      NaN      NaN
6    82070.6  45537.5 -1.18149 -3.047850    NaN       NaN      NaN      NaN
7   195049.0  28645.2 -2.60528 -1.434260      j  210687.0  26560.5  2.75992
8        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN
9        NaN      NaN      NaN       NaN    NaN       NaN      NaN      NaN

         P36 Type_10  P37  P38  P39  P40 Type_11  P41  P42  P43  P44 Type_12  \
ID
0        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
1        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
2        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
3        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
4        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
5        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
6        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
7   0.315873     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
8        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN
9        NaN     NaN  NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN     NaN

    P45  P46  P47  P48 Type_13  P49  P50  P51  P52
ID
0   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
1   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
2   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
3   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
```

```
4   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
5   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
6   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
7   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
8   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
9   NaN  NaN  NaN  NaN     NaN  NaN  NaN  NaN  NaN
```

Cada fila del dataset corresponde a un evento de colisión protón-protón. Para cada evento se conoce: - Energía faltante: - `MET`: energía transversa faltante (*missing transverse energy*) - `METphi`: ángulo azimutal asociado a la dirección de la energía faltante - Información de cada partícula reconstruida: Para cada partícula se registra: - Tipo de partícula, que puede ser: electrón, positrón, fotón, jet, b-jet, muón con carga positiva, muón con carga negativa - 4-vector, que describe la cinemática de la partícula. En este dataset, el 4-vector incluye: energía, momento transversal, pseudorapidez η y ángulo azimutal ϕ

[5]:
```
features.shape
```

[5]:
```
(5000, 67)
```

[6]:
```
y = np.genfromtxt('ParticleID_labels.txt', dtype = str)
```

[7]:
```
y
```

[7]:
```
array(['ttbar', 'ttbar', 'ttbar', …, 'ttbar', '4top', 'ttbar'],
      shape=(5000,), dtype='<U5')
```

0.0.2 Objetivo del problema

Queremos entrenar un clasificador que distinga entre dos tipos de eventos:

- `ttbar`: producción de un par top-antitop.
- `4top`: producción de cuatro quarks top.

Necesitamos transformar las dos categorías de targets desde un string a un valor numérico, como 0/1. Podemos usar la función `LabelEncoder` de sklearn

[8]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder() #cambia las categorías a 1...N
```

[9]:
```
y
```

[9]:
```
array(['ttbar', 'ttbar', 'ttbar', …, 'ttbar', '4top', 'ttbar'],
      shape=(5000,), dtype='<U5')
```

[10]:
```
y = le.fit_transform(y)
```

[11]:
```
le.classes_
```

[11]:
```
array(['4top', 'ttbar'], dtype='<U5')
```

`LabelEncoder` asigna números a las clases en el orden definido por `le.classes_`.
Como queremos que `4top` sea la clase positiva, revisamos la codificación y, si es necesario, invertimos las etiquetas.

[12]:
```
target = np.abs(y - 1)
```

[13]:
```
target # Ahora sí
```

[13]:
```
array([0, 0, 0, …, 0, 1, 0], shape=(5000,))
```

[14]:
```
y.shape
```

[14]:
```
(5000,)
```

Echemos un vistazo a las características

[15]:
```
features.describe() #Esto excluye a las columnas con datos no-numéricos
```

[15]:
```
                MET       METphi            P1            P2           P3  \
count   5000.000000  5000.000000  5.000000e+03  5.000000e+03  5000.000000
mean   64071.074332    -0.028916  3.301357e+05  1.540486e+05    -0.039812
std    60525.122480     1.819257  3.068202e+05  1.149469e+05     1.361762
min      290.756000    -3.141010  3.857940e+04  2.825400e+04    -4.110220
25%    24352.375000    -1.619905  1.369522e+05  8.883690e+04    -1.035570
50%    46814.400000    -0.055612  2.263525e+05  1.182015e+05    -0.038731
75%    83032.350000     1.537323  4.077158e+05  1.771265e+05     0.943598
max   692674.000000     3.141130  3.186360e+06  1.276710e+06     4.141410

                P4            P5            P6           P7           P8  \
count  5000.000000  4.997000e+03  4.997000e+03  4997.000000  4997.000000
mean     -0.003049  2.527799e+05  1.080302e+05    -0.029936     0.007327
std       1.814855  2.638580e+05  8.136261e+04     1.439105     1.828832
min      -3.140710  1.087540e+04  1.080000e+04    -4.668790    -3.140530
25%      -1.574213  1.007510e+05  6.321840e+04    -1.060500    -1.602460
50%      -0.009037  1.659740e+05  8.584360e+04    -0.057428     0.015111
75%       1.542370  2.999950e+05  1.238700e+05     1.028340     1.605210
max       3.138540  3.587700e+06  1.146330e+06     4.559150     3.139200

                P9           P10           P11           P12           P13  \
count  4.950000e+03   4950.000000  4950.000000  4950.000000  4.717000e+03
mean   2.117980e+05  74863.343131    -0.025104     0.011845  1.805997e+05
std    2.510361e+05  46309.512365     1.577316     1.802715  2.383403e+05
min    1.221050e+04  10639.800000    -4.520250    -3.141480  1.169190e+04
25%    7.636905e+04  46549.475000    -1.125620    -1.547418  5.999090e+04
50%    1.288565e+05  62498.400000    -0.040648     0.034238  9.922610e+04
75%    2.421225e+05  89587.500000     1.066302     1.570887  1.914340e+05
max    2.800410e+06 788338.000000     4.798090     3.139020  2.503590e+06

               P14           P15           P16           P17           P18  \
```

count	4717.000000	4717.000000	4717.000000	4.002000e+03	4002.000000
mean	57289.049481	0.010723	0.045266	1.780366e+05	48798.018516
std	32013.857623	1.634072	1.812078	2.577958e+05	26252.978520
min	10818.000000	-4.616550	-3.136130	1.110310e+04	10287.000000
25%	36097.700000	-1.121240	-1.518030	5.278370e+04	30891.650000
50%	48949.200000	-0.035512	0.060279	9.206885e+04	41054.850000
75%	68782.100000	1.159480	1.612220	1.850510e+05	58596.225000
max	481884.000000	4.730480	3.139660	3.039490e+06	331592.000000

	P19	P20	P21	P22	P23 \
count	4002.000000	4002.000000	2.871000e+03	2871.00000	2871.000000
mean	0.015167	-0.031312	1.705620e+05	44042.67015	-0.022948
std	1.744489	1.784248	2.381745e+05	23510.65367	1.806611
min	-4.778980	-3.139040	1.070330e+04	10066.90000	-4.930230
25%	-1.198468	-1.550615	5.007050e+04	28453.95000	-1.250050
50%	0.054393	-0.079641	8.593460e+04	37378.30000	-0.046667
75%	1.225223	1.508260	1.777845e+05	52731.10000	1.213660
max	4.882960	3.139910	2.791060e+06	452162.00000	4.760630

	P24	P25	P26	P27	P28 \
count	2871.000000	1.889000e+03	1889.000000	1889.000000	1889.000000
mean	0.014522	1.628825e+05	41151.069666	0.002228	0.006738
std	1.811101	2.269341e+05	20988.953157	1.815312	1.771888
min	-3.140380	1.197700e+04	11260.200000	-4.758150	-3.135630
25%	-1.586675	4.695560e+04	27963.500000	-1.231420	-1.475380
50%	0.040528	7.975460e+04	34681.700000	0.025305	0.046141
75%	1.587710	1.714090e+05	48486.200000	1.282950	1.479720
max	3.140370	1.981390e+06	293028.000000	4.781060	3.131490

	P29	P30	P31	P32	P33 \
count	1.186000e+03	1186.000000	1186.000000	1186.000000	7.290000e+02
mean	1.581409e+05	40250.387015	0.072349	-0.035907	1.596814e+05
std	2.118782e+05	26556.025657	1.836492	1.796932	2.308620e+05
min	1.380860e+04	10973.300000	-4.606330	-3.132610	1.119760e+04
25%	4.535515e+04	27140.550000	-1.243962	-1.626688	4.387110e+04
50%	8.315485e+04	33683.550000	0.156083	-0.015617	7.894980e+04
75%	1.747630e+05	45464.775000	1.380947	1.541455	1.714670e+05
max	1.932880e+06	524284.000000	4.802970	3.133340	2.111250e+06

	P34	P35	P36	P37	P38 \
count	729.000000	729.000000	729.000000	4.420000e+02	442.000000
mean	40139.289849	0.061654	-0.045868	1.574039e+05	39703.038235
std	30074.756789	1.842798	1.788596	2.165489e+05	30502.312276
min	10067.900000	-4.814380	-3.136380	1.615530e+04	10183.700000
25%	26825.000000	-1.226980	-1.513330	4.410735e+04	26589.250000
50%	33328.000000	0.072709	-0.052590	7.609735e+04	30942.700000
75%	42325.500000	1.374130	1.394730	1.837982e+05	39158.225000

```
max    341431.000000    4.372340    3.137560  1.529210e+06  369567.000000

              P39         P40           P41            P42         P43  \
count  442.000000  442.000000  2.610000e+02     261.000000  261.000000
mean     0.118543    0.024249  1.561160e+05   38173.716092    0.029455
std      1.872084    1.826435  2.319016e+05   29324.658352    1.884750
min     -4.803880   -3.135910  2.004750e+04   14800.200000   -4.400470
25%     -1.223240   -1.422415  4.092160e+04   25298.300000   -1.413650
50%      0.035675    0.090282  7.568430e+04   29479.700000   -0.088908
75%      1.532125    1.565650  1.634690e+05   36154.100000    1.416310
max      4.592710    3.138930  2.073960e+06  220722.000000    4.790720

              P44           P45            P46         P47         P48  \
count  261.000000  1.270000e+02     127.000000  127.000000  127.000000
mean     0.026422  1.631051e+05   34876.849606    0.206978   -0.001085
std      1.753017  2.248603e+05   20433.767238    1.998859    1.949004
min     -3.130690  1.780380e+04   12987.900000   -4.447660   -3.139820
25%     -1.270700  4.365005e+04   24742.500000   -1.259230   -1.817600
50%     -0.041002  8.050910e+04   28262.800000    0.120301   -0.232455
75%      1.514030  1.578350e+05   35445.700000    1.727295    1.712720
max      3.120760  1.246080e+06  167840.000000    4.691500    3.091510

              P49           P50         P51         P52
count  5.600000e+01     56.000000   56.000000   56.000000
mean   1.456600e+05  36151.183929   -0.000879    0.219260
std    1.943657e+05  25861.883410    1.941707    1.910400
min    2.512510e+04  14836.000000   -4.448760   -2.990730
25%    4.112588e+04  24974.125000   -1.243362   -1.490900
50%    9.553645e+04  27353.550000   -0.121213    0.128103
75%    1.754910e+05  33817.950000    1.800682    1.984745
max    1.177730e+06 155888.000000    4.151320    3.058890
```

[16]:
```
#features.info()
```

0.0.3 Importante:

En este dataset, los NaN aparecen porque no todos los eventos tienen la misma cantidad de partículas reconstruidas.
Al reemplazar por 0, estamos codificando implícitamente la ausencia de una partícula.

Esto puede ser razonable, pero también puede introducir una señal artificial: el modelo podría aprender no solo de las variables físicas, sino también de cuántas partículas fueron registradas.

¿Qué podemos hacer?

Opción 1: Sólo considerar las primeras 16 columnas (4 primeros productos) De esta forma, reducimos los problemas de manipulación o de imputación de datos faltantes.

Tenemos un trade-off entre tener más features, pero si tenemos más, tendremos muchos datos

faltantes que tendremos que reemplazar (imputing), o mantenemos menos features, pero el problema es más simple

[17]:
```
#limitamos los features
features_lim = features[['MET', 'METphi', 'P1', 'P2', 'P3', 'P4', 'P5', 'P6',␣
 ↪'P7', 'P8', 'P9', 'P10', 'P11',
       'P12',  'P13', 'P14', 'P15', 'P16']]
```

[18]:
```
features_lim.head()
```

[18]:
```
         MET    METphi        P1        P2        P3        P4        P5  \
ID
0   62803.5 -1.810010  137571.0  128444.0 -0.345744 -0.307112  174209.0
1   57594.2 -0.509253  161529.0   80458.3 -1.318010  1.402050  291490.0
2   82313.3  1.686840  167130.0  113078.0  0.937258 -2.068680  102423.0
3   30610.8  2.617120  112267.0   61383.9 -1.211050 -1.457800   40647.8
4   45153.1 -2.241350  178174.0  100164.0  1.166880 -0.018721   92351.3

           P6        P7        P8        P9      P10       P11       P12  \
ID
0   127932.0  0.826569  2.332000   86788.9  84554.9 -0.180795  2.187970
1    68462.9 -2.126740 -2.582310   44270.1  35139.6 -0.706120 -0.371392
2    54922.3  1.226850  0.646589   60768.9  36244.3  1.102890 -1.434480
3    39472.0 -0.024646 -2.222800  201589.0  32978.6 -2.496040  1.137810
4    69762.1  0.774114  2.568740   61625.2  50086.7  0.652572 -3.012800

          P13      P14      P15       P16
ID
0   140289.0  76955.8 -1.19933 -1.302800
1    72883.9  26902.2 -1.65386 -3.129630
2    77714.0  27801.5  1.68461  1.389690
3    90096.7  26964.5  1.87132  0.817631
4   104193.0  31151.0  1.87641  0.865381
```

[19]:
```
features_lim.describe()
```

[19]:
```
                MET       METphi            P1            P2           P3  \
count    5000.000000  5000.000000  5.000000e+03  5.000000e+03  5000.000000
mean    64071.074332    -0.028916  3.301357e+05  1.540486e+05    -0.039812
std     60525.122480     1.819257  3.068202e+05  1.149469e+05     1.361762
min       290.756000    -3.141010  3.857940e+04  2.825400e+04    -4.110220
25%     24352.375000    -1.619905  1.369522e+05  8.883690e+04    -1.035570
50%     46814.400000    -0.055612  2.263525e+05  1.182015e+05    -0.038731
75%     83032.350000     1.537323  4.077158e+05  1.771265e+05     0.943598
max    692674.000000     3.141130  3.186360e+06  1.276710e+06     4.141410

               P4            P5            P6           P7           P8  \
count  5000.000000  4.997000e+03  4.997000e+03  4997.000000  4997.000000
```

mean	-0.003049	2.527799e+05	1.080302e+05	-0.029936	0.007327
std	1.814855	2.638580e+05	8.136261e+04	1.439105	1.828832
min	-3.140710	1.087540e+04	1.080000e+04	-4.668790	-3.140530
25%	-1.574213	1.007510e+05	6.321840e+04	-1.060500	-1.602460
50%	-0.009037	1.659740e+05	8.584360e+04	-0.057428	0.015111
75%	1.542370	2.999950e+05	1.238700e+05	1.028340	1.605210
max	3.138540	3.587700e+06	1.146330e+06	4.559150	3.139200

	P9	P10	P11	P12	P13	\
count	4.950000e+03	4950.000000	4950.000000	4950.000000	4.717000e+03	
mean	2.117980e+05	74863.343131	-0.025104	0.011845	1.805997e+05	
std	2.510361e+05	46309.512365	1.577316	1.802715	2.383403e+05	
min	1.221050e+04	10639.800000	-4.520250	-3.141480	1.169190e+04	
25%	7.636905e+04	46549.475000	-1.125620	-1.547418	5.999090e+04	
50%	1.288565e+05	62498.400000	-0.040648	0.034238	9.922610e+04	
75%	2.421225e+05	89587.500000	1.066302	1.570887	1.914340e+05	
max	2.800410e+06	788338.000000	4.798090	3.139020	2.503590e+06	

	P14	P15	P16
count	4717.000000	4717.000000	4717.000000
mean	57289.049481	0.010723	0.045266
std	32013.857623	1.634072	1.812078
min	10818.000000	-4.616550	-3.136130
25%	36097.700000	-1.121240	-1.518030
50%	48949.200000	-0.035512	0.060279
75%	68782.100000	1.159480	1.612220
max	481884.000000	4.730480	3.139660

Todavía tenemos datos faltantes (valores NaN). Los reemplazaremos con 0

[20]:
```
features_lim = features_lim.fillna(0) #Llena todo lo que es NaN con 0
```

¿Qué opina de esta estrategia? Reemplazar los valores faltantes con 0 es una estrategia simple y rápida, y tiene sentido físico, ya que un 0 en la energía o en el momento de una partícula codifica su **ausencia**. Sin embargo, presenta limitaciones importantes:

- **Sesgo de multiplicidad**: el modelo puede aprender a distinguir clases no por las características físicas de las partículas, sino por *cuántas* partículas fueron reconstruidas. Los eventos `4top` suelen tener más partículas, por lo que la imputación con 0 podría introducir información implícita sobre la complejidad del evento.
- **Impacto en la escala**: al llenar con 0, las distribuciones de algunas variables quedarán bimodales (cero vs. valor real), lo que puede afectar el escalamiento y la optimización del SVM.

Una alternativa más robusta sería usar **solo las columnas sin valores faltantes** (por ejemplo, MET, METphi y las variables de las primeras partículas garantizadas), o realizar una imputación con la mediana o la media de la columna.

Si revisamos nuevamente con `describe`

[21]:
```
features_lim.describe()
```

[21]:

	MET	METphi	P1	P2	P3
count	5000.000000	5000.000000	5.000000e+03	5.000000e+03	5000.000000
mean	64071.074332	-0.028916	3.301357e+05	1.540486e+05	-0.039812
std	60525.122480	1.819257	3.068202e+05	1.149469e+05	1.361762
min	290.756000	-3.141010	3.857940e+04	2.825400e+04	-4.110220
25%	24352.375000	-1.619905	1.369522e+05	8.883690e+04	-1.035570
50%	46814.400000	-0.055612	2.263525e+05	1.182015e+05	-0.038731
75%	83032.350000	1.537323	4.077158e+05	1.771265e+05	0.943598
max	692674.000000	3.141130	3.186360e+06	1.276710e+06	4.141410

	P4	P5	P6	P7	P8
count	5000.000000	5.000000e+03	5.000000e+03	5000.000000	5000.000000
mean	-0.003049	2.526283e+05	1.079653e+05	-0.029918	0.007323
std	1.814855	2.638514e+05	8.138121e+04	1.438673	1.828283
min	-3.140710	0.000000e+00	0.000000e+00	-4.668790	-3.140530
25%	-1.574213	1.007050e+05	6.320943e+04	-1.059270	-1.599617
50%	-0.009037	1.658985e+05	8.581595e+04	-0.056810	0.012737
75%	1.542370	2.999058e+05	1.238520e+05	1.028055	1.601880
max	3.138540	3.587700e+06	1.146330e+06	4.559150	3.139200

	P9	P10	P11	P12	P13
count	5.000000e+03	5000.000000	5000.000000	5000.000000	5.000000e+03
mean	2.096800e+05	74114.709700	-0.024853	0.011727	1.703778e+05
std	2.506651e+05	46675.655162	1.569410	1.793678	2.352279e+05
min	0.000000e+00	0.000000	-4.520250	-3.141480	0.000000e+00
25%	7.488228e+04	46165.375000	-1.108390	-1.532478	5.480870e+04
50%	1.277135e+05	62167.100000	-0.023321	0.006687	9.259335e+04
75%	2.406498e+05	89065.300000	1.048617	1.553310	1.831228e+05
max	2.800410e+06	788338.000000	4.798090	3.139020	2.503590e+06

	P14	P15	P16
count	5000.000000	5000.000000	5000.000000
mean	54046.489280	0.010116	0.042704
std	33795.723384	1.587146	1.760070
min	0.000000	-4.616550	-3.136130
25%	33959.400000	-1.050477	-1.424080
50%	47278.800000	0.000000	0.000000
75%	66846.300000	1.085627	1.521765
max	481884.000000	4.730480	3.139660

Ahora tenemos tamaños consistentes para poder usarlos como features, PERO hay que estar conciente de los efectos negativos que puede traer nuestra estrategia de imputación

0.1 Benchmark inicial

En este notebook, el benchmark principal es el desempeño de estrategias simples antes de usar un SVM más flexible. Primero comparamos contra un clasificador que siempre predice la clase mayoritaria, ya que el dataset está desbalanceado. Luego, un clasificador aleatorio, es decir, asigna un valor aleatorio a cada clase. Y también usamos un `LinearSVC` con escalamiento como benchmark lineal: una referencia razonable para saber si necesitamos modelos más complejos, como SVM con kernels no lineales.

[22]:
```
np.sum(target)/len(target) #distribución
```

[22]:
```
np.float64(0.1622)
```

84% en la etiqueta negativa, 16% en la etiqueta positiva.

Esto significa que un clasificador que pone todo en la clase negativa (lazy classifier) tendrá un 84% de accuracy

Cómo lo haría un clasificador aleatorio que asigna un valor aleatorio a la distribución de clase?

[23]:
```
#Numerical solution

acc=0
for i in range(1000): #1000 iteraciones del proceso
    x = np.random.choice(target,5000) #generamos 5000 etiquetas aleatorias
    acc += metrics.accuracy_score(target,x) #calculamos accuracy promedio en␣
 ↪las 1000 iteraciones
print(acc/1000)

#Analytic solution
#P(Clase 1)^2 + P(Clase 2)^2 exactitud esperada


print(0.8378*(0.8378) + 0.1622*0.1622)
```

```
0.7283292000000001
0.72821768
```

En conclusión, un clasificador aleatorio tendrá 73% accuracy, un lazy classfier tiene 83% accuracy. Este análisis es útil para tener una expectativa de qué es un "buen resultado" y qué significa una mejora significativa

0.1.1 Ahora empezaremos con un modelo lineal model = SVC()

Definimos una estrategia CV, establecemos un benchmark para un modelo lineal no regularization (parámetro C muy alto)

Antes de correr el modelo, discuta:

1. ¿Esperamos que un modelo lineal separe bien eventos `ttbar` y `4top`?
2. ¿Qué significaría que el modelo lineal tenga una accuracy cercana al clasificador mayoritario?

3. ¿Qué métrica sería más relevante si queremos detectar eventos `4top`: accuracy, precision, recall o F1-score?

[24]:
```
bmodel = LinearSVC(dual = False, C=1000)
# Para modelo lineal, se prefiere dual=False cuando  n_samples > n_features.
#( el primal trabaja con las features, el dual trabaja con los alpha_i que␣
 ↪estan asociados al numero de muestras)
# Si no, no va a converger!
```

¿Qué significa la elección de C grande?

El parámetro C controla el trade-off entre maximizar el margen y minimizar las violaciones de margen (errores de clasificación). Un valor de C muy grande (como $C = 1000$) penaliza fuertemente los errores: el modelo intenta clasificar correctamente *todos* los puntos de entrenamiento, reduciendo el margen. Esto equivale a casi **sin regularización**, y puede llevar a sobreajuste (*overfitting*).

[25]:
```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=101)
```

[26]:
```
l_benchmark_lim = cross_validate(bmodel, features_lim, target, cv = cv, scoring␣
 ↪= 'accuracy', return_train_score=True)
```

[27]:
```
l_benchmark_lim
```

[27]:
```
{'fit_time': array([0.02778578, 0.01566982, 0.02133441, 0.02016449,
0.01438236]),
 'score_time': array([0.00343657, 0.00211859, 0.0033617 , 0.00269747,
0.00193858]),
 'test_score': array([0.841, 0.825, 0.829, 0.83 , 0.833]),
 'train_score': array([0.8315 , 0.83275, 0.8315 , 0.83125, 0.832  ])}
```

[28]:
```
np.round(l_benchmark_lim['test_score'].mean(),3), np.
 ↪round(l_benchmark_lim['test_score'].std(), 3)
```

[28]:
```
(np.float64(0.832), np.float64(0.005))
```

El promedio de accuracy para este caso parece alto, pero es igual de alto que el lazy classifier. Podemos chequear las etiquetas con `cross_val_predict`, que compila las etiquetas predichas cuando cada objeto estaba en el fold test

[29]:
```
ypred_bench_lim = cross_val_predict(bmodel, features_lim, target, cv = cv)
```

[30]:
```
ypred_bench_lim
```

[30]:
```
array([0, 0, 0, …, 0, 0, 0], shape=(5000,))
```

0.1.2 Pregunta: Qué podríamos haber hecho antes de haber construído el modelo SVM?

Mostrar explicación

0.1.3 Escalar!

Los SVM son sensibles a la escala de las variables porque trabajan en el espacio de características. La optimización depende de productos punto entre pares de ejemplos:

$$\mathbf{x}_i \cdot \mathbf{x}_j$$

Por eso escalamos las features antes de entrenar un SVM. Una opción común es usar `StandardScaler`, que transforma cada feature para que tenga media 0 y desviación estándar 1.

[31]:
```
from sklearn.pipeline import make_pipeline #Podemos construir varios pasos al␣
↪mismo tiempo
```

¿Por qué usamos Pipeline?

Porque en validación cruzada, el escalamiento debe aprenderse usando solo los datos de entrenamiento de cada fold.
Si escalamos todo el dataset antes de hacer CV, estaríamos usando información del conjunto de prueba para transformar los datos.
Eso sería una forma sutil de data leakage.

`make_pipeline` permite encadenar varios pasos de preprocesamiento y modelado en un solo objeto.

[32]:
```
piped_model = make_pipeline(StandardScaler(), LinearSVC(dual = False, C = 1000))
#modelo benchmark (de referencia)
benchmark_lim_piped = cross_validate(piped_model, features_lim, target, cv =␣
↪cv, scoring = 'accuracy', return_train_score=True)
```

[33]:
```
benchmark_lim_piped
```

[33]:
```
{'fit_time': array([0.0116992 , 0.01252198, 0.01060891, 0.00944829,
0.00990486]),
 'score_time': array([0.00219822, 0.0025773 , 0.00225687, 0.00188255,
0.00198102]),
 'test_score': array([0.894, 0.889, 0.89 , 0.892, 0.899]),
 'train_score': array([0.89575, 0.895  , 0.895  , 0.89825, 0.89275])}
```

[34]:
```
np.round(benchmark_lim_piped['test_score'].mean(),3), np.
↪round(benchmark_lim_piped['test_score'].std(), 3)
```

[34]:
```
(np.float64(0.893), np.float64(0.004))
```

[35]:
```
np.round(benchmark_lim_piped['train_score'].mean(),3), np.
↪round(benchmark_lim_piped['train_score'].std(), 3)
```

[35]:
```
(np.float64(0.895), np.float64(0.002))
```

Esto es una mejora!

Ahora necesitamos **diagnosticar** nuestro modelo:

Si comparamos el score para prueba y entrenamiento, notamos que hay un problema.... Miremos la curva de aprendizaje, que nos ayudará a mirar esto mejor y nos indicará si necesitamos más datos

0.1.4 Curva de aprendizaje

[36]:
```
from sklearn.model_selection import learning_curve

def plot_learning_curve(estimator, title, X, y, ylim=None, cv=5,
                        n_jobs=-1, train_sizes=np.linspace(.1, 1.0, 5), scoring␣
 ↪= 'accuracy', scale = False):
    """
    Generate a simple plot of the test and training learning curve.

    Parameters
    ----------
    estimator : object type that implements the "fit" and "predict" methods
        An object of that type which is cloned for each validation.

    title : string
        Title for the chart.

    X : array-like, shape (n_samples, n_features)
        Training vector, where n_samples is the number of samples and
        n_features is the number of features.

    y : array-like, shape (n_samples) or (n_samples, n_features), optional
        Target relative to X for classification or regression;
        None for unsupervised learning.

    ylim : tuple, shape (ymin, ymax), optional
        Defines minimum and maximum yvalues plotted.

    cv : int, cross-validation generator or an iterable, optional
        Determines the cross-validation splitting strategy.
        Possible inputs for cv are:
          - None, to use the default 3-fold cross-validation,
          - integer, to specify the number of folds.
          - :term:`CV splitter`,
          - An iterable yielding (train, test) splits as arrays of indices.

        For integer/None inputs, if ``y`` is binary or multiclass,
        :class:`StratifiedKFold` used. If the estimator is not a classifier
        or if ``y`` is neither binary nor multiclass, :class:`KFold` is used.

        Refer :ref:`User Guide <cross_validation>` for the various
        cross-validators that can be used here.
```

```
    n_jobs : int or None, optional (default=None)
        Number of jobs to run in parallel.
        ``None`` means 1 unless in a :obj:`joblib.parallel_backend` context.
        ``-1`` means using all processors. See :term:`Glossary <n_jobs>`
        for more details.

    train_sizes : array-like, shape (n_ticks,), dtype float or int
        Relative or absolute numbers of training examples that will be used to
        generate the learning curve. If the dtype is float, it is regarded as a
        fraction of the maximum size of the training set (that is determined
        by the selected validation method), i.e. it has to be within (0, 1].
        Otherwise it is interpreted as absolute sizes of the training sets.
        Note that for classification the number of samples usually have to
        be big enough to contain at least one sample from each class.
        (default: np.linspace(0.1, 1.0, 5))
    """
    plt.figure()
    plt.title(title)
    if ylim is not None:
        plt.ylim(*ylim)
    plt.xlabel("# of training examples",fontsize = 14)

    plt.ylabel("Accuracy score",fontsize = 14)

    if (scale == True):
        scaler = sklearn.preprocessing.StandardScaler()
        X = scaler.fit_transform(X)

    train_sizes, train_scores, test_scores = learning_curve(
        estimator, X, y, cv=cv, n_jobs=n_jobs, train_sizes=train_sizes, scoring␣
 ↪= scoring)
    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
    test_scores_std = np.std(test_scores, axis=1)
#    plt.grid()

    plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                     train_scores_mean + train_scores_std, alpha=0.1,
                     color="b")
    plt.fill_between(train_sizes, test_scores_mean - test_scores_std,
                     test_scores_mean + test_scores_std, alpha=0.1, color="g")
    plt.plot(train_sizes, train_scores_mean, 'o-', color="b",
             label="Training score from CV")
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g",
             label="Test score from CV")
```

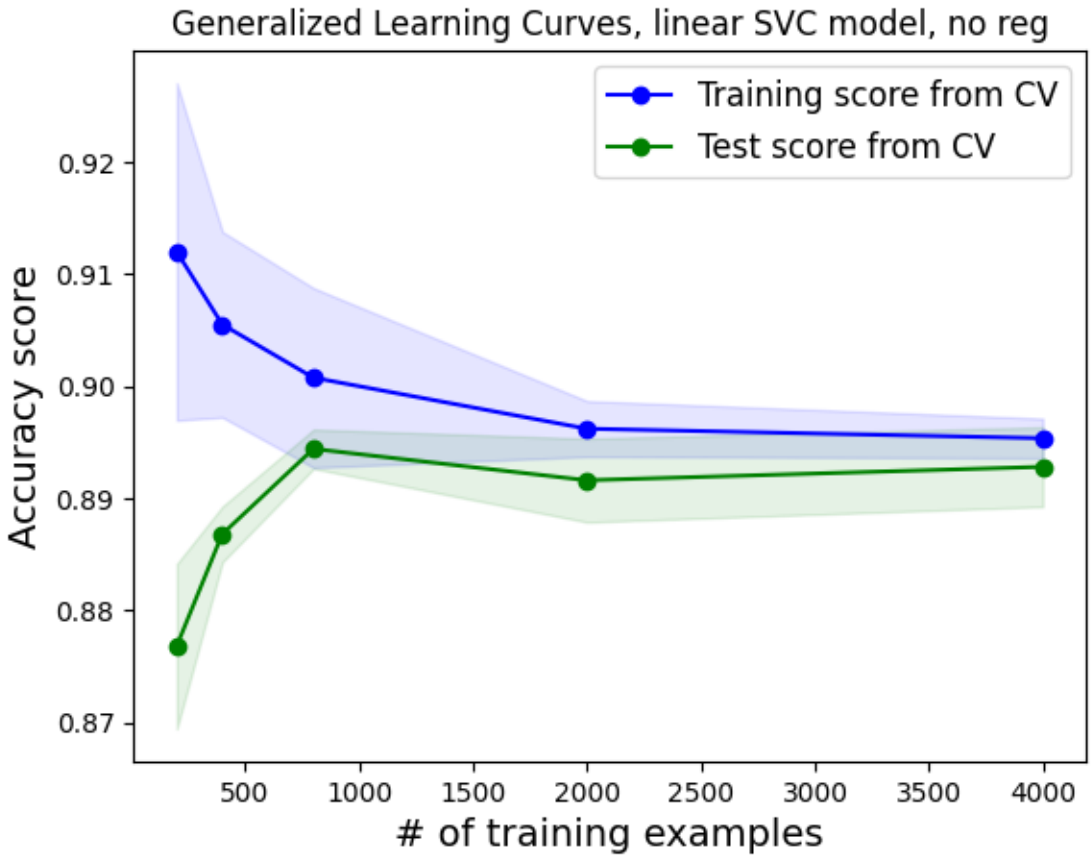
```
    plt.legend(loc="best",fontsize = 12)
    return plt
```

[37]:
```
cv
```

[37]:
```
StratifiedKFold(n_splits=5, random_state=101, shuffle=True)
```

[38]:
```
plot_learning_curve(piped_model, 'Generalized Learning Curves, linear SVC␣
↪model, no reg',
                    features_lim, target, train_sizes = np.array([0.05,0.1,0.
↪2,0.5,1.0]), cv = cv)
```

[38]:
```
<module 'matplotlib.pyplot' from
'/home/jorgeluis/miniconda3/lib/python3.13/site-packages/matplotlib/pyplot.py'>
```



[39]:
```
# Obtener predicciones usando validación cruzada
ypred_linear = cross_val_predict(
    piped_model,
    features_lim,
```

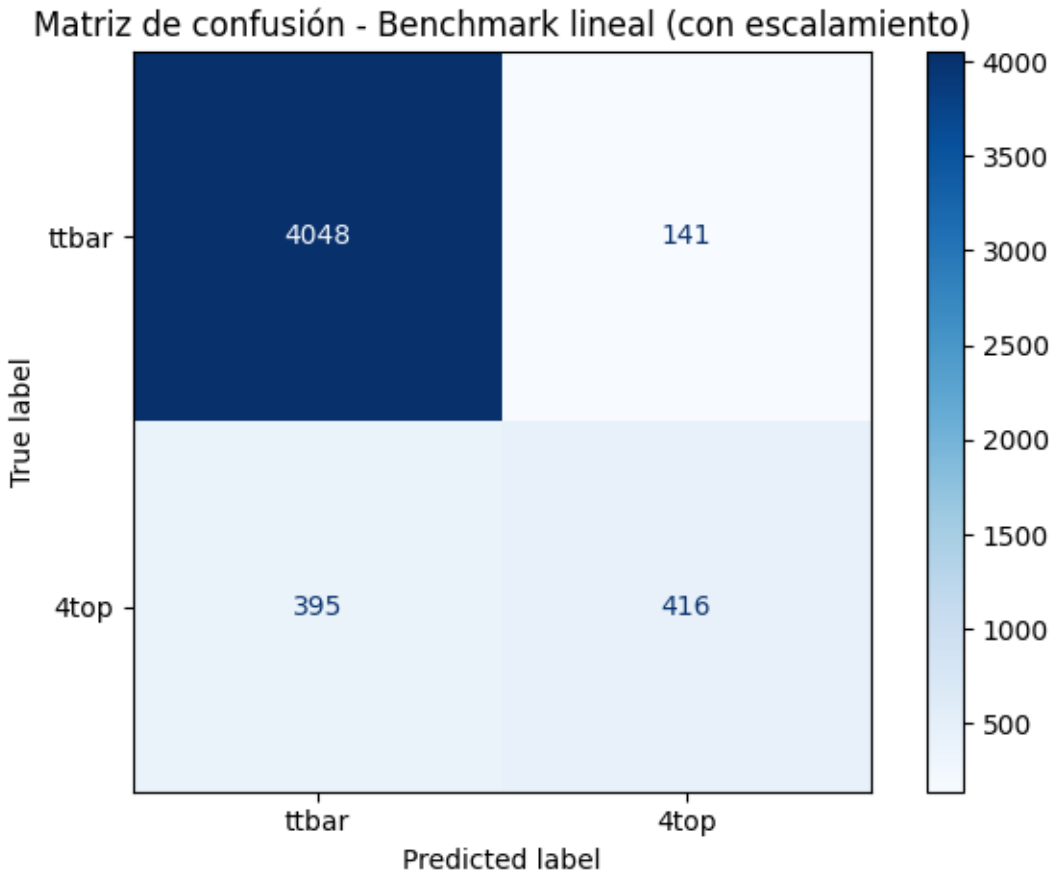
```
    target,
    cv=cv
)
```

Genere una matriz de confusión para esre modelo y evalúe el desempeño del modelo a partir del resultado. también puede usar la función `classification_report` de `sklearn`

[40]:
```
# Matriz de confusión para el modelo lineal con escalamiento
from sklearn.metrics import confusion_matrix, classification_report,␣
 ↪ConfusionMatrixDisplay

cm_linear = confusion_matrix(target, ypred_linear)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_linear,␣
 ↪display_labels=['ttbar', '4top'])
disp.plot(cmap='Blues')
plt.title('Matriz de confusión - Benchmark lineal (con escalamiento)')
plt.tight_layout()
plt.show()

print(classification_report(target, ypred_linear, target_names=['ttbar',␣
 ↪'4top']))
```



	precision	recall	f1-score	support
ttbar	0.91	0.97	0.94	4189
4top	0.75	0.51	0.61	811
accuracy			0.89	5000
macro avg	0.83	0.74	0.77	5000
weighted avg	0.88	0.89	0.88	5000

0.2 Conclusiones del benchmark lineal

1. En la curva de aprendizaje, ¿los scores de entrenamiento y validación son altos o bajos? ¿Están separados o son parecidos?
2. ¿La curva sugiere alto bias o alta variance?
   Justifique usando la relación entre score de entrenamiento y validación.
3. Considerando que usamos (C=1000), ¿qué nos dice este resultado sobre la capacidad del modelo lineal?
4. En la matriz de confusión, ¿el modelo detecta bien los eventos `4top` o tiende a clasificarlos como `ttbar`?
5. ¿qué métrica muestra mejor el problema con la clase minoritaria: accuracy, precision, recall o F1-score?
6. ¿El modelo lineal supera realmente los benchmarks ingenuos, o su buen desempeño se explica principalmente por el desbalance de clases?
7. ¿Qué evidencia nos indica que necesitamos un modelo más flexible que una frontera lineal?

0.2.1 Respuestas: Conclusiones del benchmark lineal

1. **¿Los scores de entrenamiento y validación son altos o bajos? ¿Están separados o son parecidos?**
   El score de entrenamiento (~0.895) y el de validación (~0.893) son muy similares y relativamente altos en accuracy. Sin embargo, la accuracy refleja principalmente la clase mayoritaria (`ttbar`). Al revisar el recall de `4top` (~0.51), el modelo detecta solo la mitad de los eventos positivos.
2. **¿La curva sugiere alto bias o alta variance?**
   Ambas curvas convergen a un nivel similar pero no muy alto para `4top`. La brecha pequeña entre train y test indica **alto bias** (underfitting), no alta variance. El modelo lineal no es suficientemente expresivo para capturar la frontera de decisión real.
3. **Con $C = 1000$, ¿qué nos dice este resultado sobre el modelo lineal?**
   Incluso con prácticamente sin regularización ($C = 1000$), el modelo lineal no mejora significativamente. Esto indica que el problema **no es linealmente separable** con estas features: el límite que necesitamos es inherentemente no lineal.

4. **¿El modelo detecta bien los eventos `4top` o tiende a clasificarlos como `ttbar`?**
   El modelo tiene un recall de ~0.51 para `4top`, lo que significa que casi la mitad de los eventos positivos son clasificados incorrectamente como `ttbar`. La matriz de confusión muestra ~395 falsos negativos frente a ~416 verdaderos positivos.

5. **¿Qué métrica muestra mejor el problema con la clase minoritaria?**
   El **recall** (0.51) y el **F1-score** (0.61) son las métricas más informativas, ya que la accuracy (89%) es engañosa: se explica en gran parte por el correcto rechazo de la clase mayoritaria `ttbar`.

6. **¿El modelo lineal supera realmente los benchmarks ingenuos?**
   En accuracy, sí supera tanto al clasificador aleatorio (73%) como al lazy classifier (83%). Sin embargo, el lazy classifier tiene un recall de 0% para `4top`, mientras que el modelo lineal tiene 51%. La mejora es real, pero modesta para la clase de interés.

7. **¿Qué evidencia indica que necesitamos un modelo más flexible?**
   La convergencia a valores bajos de recall para `4top` incluso con C alto, y el hecho de que el modelo lineal no mejora al agregar más regularización, indican que la frontera de decisión real es **no lineal**. Es necesario un kernel no lineal (por ejemplo, RBF) para capturar la estructura del espacio de features.

0.3 Optimización de hiperparámetros

`GridSearchCV`selecciona los hiperparámetros que obtienen el mejor desempeño promedio en validación cruzada. Sin embargo, ese score puede ser optimista, porque los mismos folds de validación se usaron para elegir la mejor combinación de hiperparámetros. Para estimar mejor el desempeño esperado en datos nuevos, se puede usar **validación cruzada anidada**. En este procedimiento, una validación cruzada interna selecciona los hiperparámetros, mientras que una validación cruzada externa evalúa el modelo seleccionado. La idea central es no usar la misma información para elegir el modelo y para reportar su desempeño final.

La validación cruzada anidada es más rigurosa, pero no siempre vale la pena implementarla. Puede ser muy costosa, porque requiere repetir la búsqueda de hiperparámetros dentro de cada fold externo.En este notebook usaremos `GridSearchCV` para seleccionar hiperparámetros y comparar modelos, pero debemos recordar que el mejor score obtenido puede ser algo optimista. Si el objetivo fuera reportar un resultado final de investigación de manera rigurosa, sería mejor usar un conjunto de prueba independiente o validación cruzada anidada.

[41]:
```
#ahora usamos la version general SVC y no la lineal, para cambiar el kernel
piped_model = make_pipeline(StandardScaler(), SVC())

piped_model.get_params() # esto nos muestra los parámetros para el scaler y el␣
 ↪clasificador
```

[41]:
```
{'memory': None,
 'steps': [('standardscaler', StandardScaler()), ('svc', SVC())],
 'transform_input': None,
 'verbose': False,
 'standardscaler': StandardScaler(),
 'svc': SVC(),
```

```
 'standardscaler__copy': True,
 'standardscaler__with_mean': True,
 'standardscaler__with_std': True,
 'svc__C': 1.0,
 'svc__break_ties': False,
 'svc__cache_size': 200,
 'svc__class_weight': None,
 'svc__coef0': 0.0,
 'svc__decision_function_shape': 'ovr',
 'svc__degree': 3,
 'svc__gamma': 'scale',
 'svc__kernel': 'rbf',
 'svc__max_iter': -1,
 'svc__probability': False,
 'svc__random_state': None,
 'svc__shrinking': True,
 'svc__tol': 0.001,
 'svc__verbose': False}
```

0.3.1 Podemos definir un diccionario de valores parámetros para correr la optimización

Note que esto tomará tiempo (~3 min en mi computador)

Una vez que corramos esta celda, el objeto modelo tendra atributos de "best_score_", "best_params_" y "best_estimator_", que nos da acceso al estimador óptimo y "cv_results_" que pueden ser utilizados para visualizar la performance de todos los modelo

[42]:
```
%%time

parameters = {'svc__kernel':['poly', 'rbf'], \
              'svc__gamma':[0.00001,'scale', 0.01, 0.1], 'svc__C':[0.1, 1.0, 10.
 ↪0, 100.0, 1000], \
              'svc__degree': [2, 4, 8]}

model = GridSearchCV(piped_model, parameters, cv=StratifiedKFold(n_splits=5,␣
 ↪shuffle=True),
                     verbose=2, n_jobs=4, return_train_score=True)
model.fit(features_lim, target)

print('Best params, best score:', "{:.4f}".format(model.best_score_),
      model.best_params_)
```

```
Fitting 5 folds for each of 120 candidates, totalling 600 fits
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
```

```
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
```

```
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
```

```
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
```

```
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=0.1, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
```

```
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=0.1, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
```

```
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
```

```
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
```

```
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
```

```
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
```

```
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.4s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.3s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
```

```
[CV] END svc__C=1.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total time=
0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
```

```
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   1.6s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   1.4s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   1.8s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   1.7s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=   2.1s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
```

```
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
```

```
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
```

```
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
```

```
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=10.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   4.9s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   5.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   5.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   6.0s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
```

```
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=   5.9s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=  15.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=  18.2s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=  18.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=  20.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
```

```
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.5s
```

```
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
```

```
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=100.0, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
```

```
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=100.0, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time=  20.5s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=  53.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time=  55.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=rbf; total
time=   0.5s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   1.5s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time= 1.1min
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   2.1s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   1.8s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   2.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=poly; total
time=   2.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   1.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   1.6s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time= 1.2min
```

```
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.0s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   1.9s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.1s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=scale, svc__kernel=poly; total
time= 1.1min
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time= 2.5min
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time= 3.0min
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   1.1s
```

```
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.6s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   3.1s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time= 3.5min
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.7s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.01, svc__kernel=rbf; total
time=   3.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.8s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   1.1s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
```

```
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=4, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=poly; total
time=   0.2s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=1e-05, svc__kernel=rbf; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   1.1s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   1.1s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=scale, svc__kernel=rbf; total
time=   1.0s
```

```
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.4s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=poly; total
time=   0.3s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time= 3.1min
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   3.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.8s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.5s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.01, svc__kernel=rbf; total
time=   2.7s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   1.0s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=poly; total
time=   0.9s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.6s
[CV] END svc__C=1000, svc__degree=8, svc__gamma=0.1, svc__kernel=rbf; total
time=   0.7s
[CV] END svc__C=1000, svc__degree=2, svc__gamma=0.1, svc__kernel=poly; total
time= 3.6min
Best params, best score: 0.8938 {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma':
'scale', 'svc__kernel': 'rbf'}
CPU times: user 2.96 s, sys: 509 ms, total: 3.47 s
Wall time: 8min 44s
```

Ahora, visualizamos los modelos en un dataframe, y los rankeamos de acuerdo a los scores de prueba. Miraremos el promedio y desviación estándar de los test scores, el promedio de los train scores, para evaluar si son distintos y la significancia del resultado, y fitting time, para elegir el modelo más rápido en vez del mejor modelo, si es que los scores son comparables

[43]:
```
scores_lim = pd.DataFrame(model.cv_results_)

scores_lim.columns
```

[43]:
```
Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
       'param_svc__C', 'param_svc__degree', 'param_svc__gamma',
       'param_svc__kernel', 'params', 'split0_test_score', 'split1_test_score',
       'split2_test_score', 'split3_test_score', 'split4_test_score',
       'mean_test_score', 'std_test_score', 'rank_test_score',
       'split0_train_score', 'split1_train_score', 'split2_train_score',
       'split3_train_score', 'split4_train_score', 'mean_train_score',
       'std_train_score'],
      dtype='str')
```

[44]:
```
scores_lim[['params','mean_test_score','std_test_score','mean_train_score', \
            'mean_fit_time']].sort_values(by = 'mean_test_score', ascending =␣
 ↪False)
```

[44]:
```
params  \
35      {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
43      {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
27      {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
53       {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
69       {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
61       {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
45        {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
29        {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
97      {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
105     {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
37       {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
113     {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
```

```
39          {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
47          {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
31          {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
93       {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
77       {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
85       {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
3       {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
11      {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
19      {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
89      {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
81      {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
73      {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
13         {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
21         {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
5          {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
51     {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
67     {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
59     {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
63         {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
71         {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
55         {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
100      {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
54        {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
102       {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
```

```
'poly'}
76     {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
78      {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
98    {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
50    {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
30         {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
74   {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
101       {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
117       {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
109       {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'rbf'}
26     {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
6          {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
52       {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
14         {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
34     {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
108      {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
58    {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
38         {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
79        {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
95        {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
87        {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
2      {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
111        {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
103        {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
```

```
119       {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
42     {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
22        {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
84      {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
66    {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
10     {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
28       {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
91    {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
83    {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
75    {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
18     {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
90   {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
46        {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
23         {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
15         {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
7          {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'rbf'}
115    {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
99     {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
107    {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'rbf'}
60      {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
62       {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
82   {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
116     {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
92     {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
```

```
'poly'}
36       {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
4        {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
68      {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
110      {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
106   {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
86      {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
49      {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
57      {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
65      {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
44       {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
118      {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
12       {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
20       {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
114   {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
70       {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
9        {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
0       {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
16      {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
8       {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
1        {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
17       {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
48     {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
41       {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
```

```
40      {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
33       {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
25       {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'rbf'}
24      {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
32      {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
64     {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
80    {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
72    {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
56     {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
88    {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
104    {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
96     {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
112    {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
94      {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}

     mean_test_score  std_test_score  mean_train_score  mean_fit_time
35            0.8938        0.005741           0.92225       0.165597
43            0.8938        0.005741           0.92225       0.154997
27            0.8938        0.005741           0.92225       0.160769
53            0.8932        0.005344           0.91300       0.150383
69            0.8932        0.005344           0.91300       0.150248
61            0.8932        0.005344           0.91300       0.152499
45            0.8930        0.006356           0.90035       0.120449
29            0.8930        0.006356           0.90035       0.153501
97            0.8930        0.008124           0.89445       0.228507
105           0.8930        0.008124           0.89445       0.120479
37            0.8930        0.006356           0.90035       0.135687
113           0.8930        0.008124           0.89445       0.213035
39            0.8928        0.006177           0.93935       0.178703
47            0.8928        0.006177           0.93935       0.210065
31            0.8928        0.006177           0.93935       0.203011
93            0.8924        0.006151           0.93325       0.387731
77            0.8924        0.006151           0.93325       0.360974
```

85	0.8924	0.006151	0.93325	0.404438
3	0.8844	0.005748	0.88995	0.159072
11	0.8844	0.005748	0.88995	0.156457
19	0.8844	0.005748	0.88995	0.149785
89	0.8844	0.005389	0.88560	0.134808
81	0.8844	0.005389	0.88560	0.124374
73	0.8844	0.005389	0.88560	0.127922
13	0.8830	0.004561	0.88475	0.142574
21	0.8830	0.004561	0.88475	0.134849
5	0.8830	0.004561	0.88475	0.140675
51	0.8794	0.007446	0.96680	0.255019
67	0.8794	0.007446	0.96680	0.224039
59	0.8794	0.007446	0.96680	0.226134
63	0.8790	0.011489	0.99090	0.267841
71	0.8790	0.011489	0.99090	0.262869
55	0.8790	0.011489	0.99090	0.295223
100	0.8778	0.004707	0.88660	2.068984
54	0.8778	0.004665	0.88660	1.692990
102	0.8772	0.003709	0.88710	188.422067
76	0.8770	0.004382	0.88450	0.347565
78	0.8770	0.004000	0.88740	18.519309
98	0.8770	0.003950	0.88720	62.355202
50	0.8770	0.004336	0.88545	0.685815
30	0.8770	0.004382	0.88445	0.339399
74	0.8768	0.003868	0.88720	5.431899
101	0.8766	0.008593	0.96320	1.814657
117	0.8766	0.008593	0.96320	2.636499
109	0.8766	0.008593	0.96320	2.778801
26	0.8764	0.003200	0.88210	0.223022
6	0.8738	0.002993	0.87800	0.173279
52	0.8738	0.002993	0.87810	0.154187
14	0.8718	0.005381	0.91735	0.257908
34	0.8718	0.005381	0.91665	0.283627
108	0.8718	0.005381	0.91735	0.452844
58	0.8700	0.007239	0.96085	0.398285
38	0.8684	0.008754	0.96205	0.392703
79	0.8682	0.006615	1.00000	0.324405
95	0.8682	0.006615	1.00000	0.447380
87	0.8682	0.006615	1.00000	0.311696
2	0.8678	0.001470	0.87165	0.141634
111	0.8674	0.006974	1.00000	0.585492
103	0.8674	0.006974	1.00000	0.322049
119	0.8674	0.006974	1.00000	0.568208
42	0.8664	0.004030	0.91605	0.346007
22	0.8664	0.006651	0.94505	0.577502
84	0.8658	0.003187	0.88585	0.193432
66	0.8658	0.006242	0.94380	0.575793

10	0.8654	0.003262	0.88550	0.197970
28	0.8654	0.001960	0.86650	0.139850
91	0.8638	0.008060	0.99675	0.478389
83	0.8638	0.008060	0.99675	0.441921
75	0.8638	0.008060	0.99675	0.465753
18	0.8638	0.001720	0.90085	0.256716
90	0.8632	0.009108	0.97240	0.681027
46	0.8628	0.009368	0.97325	0.695067
23	0.8602	0.004020	0.86645	0.171519
15	0.8602	0.004020	0.86645	0.176708
7	0.8602	0.004020	0.86645	0.170955
115	0.8598	0.006462	1.00000	0.896005
99	0.8598	0.006462	1.00000	0.540818
107	0.8598	0.006462	1.00000	0.845029
60	0.8566	0.003720	0.86440	0.159702
62	0.8546	0.004454	0.99645	0.571410
82	0.8546	0.005083	0.99620	0.508218
116	0.8524	0.003720	0.86955	0.335029
92	0.8516	0.004716	0.85915	0.167711
36	0.8500	0.003033	0.85280	0.172879
4	0.8486	0.003137	0.84950	0.142473
68	0.8478	0.003124	0.85250	0.180190
110	0.8470	0.007266	1.00000	0.859108
106	0.8466	0.008015	1.00000	0.749852
86	0.8462	0.008280	1.00000	0.487257
49	0.8438	0.003059	0.84430	0.148404
57	0.8438	0.003059	0.84430	0.141244
65	0.8438	0.003059	0.84430	0.135920
44	0.8422	0.001470	0.84460	0.151117
118	0.8408	0.007359	1.00000	0.861196
12	0.8408	0.001166	0.84215	0.180968
20	0.8402	0.001833	0.84175	0.161869
114	0.8390	0.012633	0.99530	0.924989
70	0.8382	0.012287	0.99575	0.562088
9	0.8378	0.000400	0.83780	0.142411
0	0.8378	0.000400	0.83780	0.110095
16	0.8378	0.000400	0.83780	0.117666
8	0.8378	0.000400	0.83780	0.108694
1	0.8378	0.000400	0.83780	0.143521
17	0.8378	0.000400	0.83780	0.153926
48	0.8378	0.000400	0.83780	0.101024
41	0.8378	0.000400	0.83780	0.164765
40	0.8378	0.000400	0.83780	0.103711
33	0.8378	0.000400	0.83780	0.138863
25	0.8378	0.000400	0.83780	0.159414
24	0.8378	0.000400	0.83780	0.119441
32	0.8378	0.000400	0.83780	0.111129

```
64          0.8378        0.000400          0.83780        0.101890
80          0.8378        0.000400          0.83780        0.102557
72          0.8378        0.000400          0.83780        0.097364
56          0.8378        0.000400          0.83780        0.106867
88          0.8378        0.000400          0.83780        0.099210
104         0.8378        0.000400          0.83780        0.101397
96          0.8378        0.000400          0.83780        0.141612
112         0.8378        0.000400          0.83780        0.179327
94          0.8362        0.007359          0.99960        0.616119
```

Observamos que el kernel rbf es preferido constantemente, los valores de C y γ no afectan demasiado los scores. GridSearch es insensible a combinaciones de parámetros irrelevantes, por ejemplo, los tres primeros modelos son idénticos porque el grado del kernel polinomial no afecta al kernel rbf.

Podemos aislar un solo tipo de kernel

[45]:
```
scores_lim[scores_lim['param_svc__kernel'] ==␣
↪'poly'][['params','mean_test_score','std_test_score',\
                        'mean_train_score','mean_fit_time']].sort_values(by =␣
↪'mean_test_score', ascending = False)
```

[45]:
```
params  \
54        {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
100      {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
102       {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
30         {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
78       {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
76      {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
50    {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
98    {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
74   {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
26     {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
6          {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
52       {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
34     {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
```

```
108     {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
14        {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
58    {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
38        {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
2     {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
42    {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
22        {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
66    {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
84     {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
28       {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
10    {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
18    {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
90   {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
46        {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
60      {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
62       {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
82   {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
116     {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
92     {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
36       {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
4        {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
68      {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
110      {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
106   {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 'scale', 'svc__kernel':
```

```
'poly'}
86       {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
44        {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
12        {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
118       {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
20        {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 0.01, 'svc__kernel':
'poly'}
114   {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 'scale', 'svc__kernel':
'poly'}
70        {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
48      {'svc__C': 10.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
40       {'svc__C': 1.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
24       {'svc__C': 1.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
32       {'svc__C': 1.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
8        {'svc__C': 0.1, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
0        {'svc__C': 0.1, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
16       {'svc__C': 0.1, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
64      {'svc__C': 10.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
80     {'svc__C': 100.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
72     {'svc__C': 100.0, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
56      {'svc__C': 10.0, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
88     {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
104     {'svc__C': 1000, 'svc__degree': 4, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
96      {'svc__C': 1000, 'svc__degree': 2, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
112     {'svc__C': 1000, 'svc__degree': 8, 'svc__gamma': 1e-05, 'svc__kernel':
'poly'}
94        {'svc__C': 100.0, 'svc__degree': 8, 'svc__gamma': 0.1, 'svc__kernel':
'poly'}
```

	mean_test_score	std_test_score	mean_train_score	mean_fit_time
54	0.8778	0.004665	0.88660	1.692990
100	0.8778	0.004707	0.88660	2.068984
102	0.8772	0.003709	0.88710	188.422067
30	0.8770	0.004382	0.88445	0.339399
78	0.8770	0.004000	0.88740	18.519309
76	0.8770	0.004382	0.88450	0.347565
50	0.8770	0.004336	0.88545	0.685815
98	0.8770	0.003950	0.88720	62.355202
74	0.8768	0.003868	0.88720	5.431899
26	0.8764	0.003200	0.88210	0.223022
6	0.8738	0.002993	0.87800	0.173279
52	0.8738	0.002993	0.87810	0.154187
34	0.8718	0.005381	0.91665	0.283627
108	0.8718	0.005381	0.91735	0.452844
14	0.8718	0.005381	0.91735	0.257908
58	0.8700	0.007239	0.96085	0.398285
38	0.8684	0.008754	0.96205	0.392703
2	0.8678	0.001470	0.87165	0.141634
42	0.8664	0.004030	0.91605	0.346007
22	0.8664	0.006651	0.94505	0.577502
66	0.8658	0.006242	0.94380	0.575793
84	0.8658	0.003187	0.88585	0.193432
28	0.8654	0.001960	0.86650	0.139850
10	0.8654	0.003262	0.88550	0.197970
18	0.8638	0.001720	0.90085	0.256716
90	0.8632	0.009108	0.97240	0.681027
46	0.8628	0.009368	0.97325	0.695067
60	0.8566	0.003720	0.86440	0.159702
62	0.8546	0.004454	0.99645	0.571410
82	0.8546	0.005083	0.99620	0.508218
116	0.8524	0.003720	0.86955	0.335029
92	0.8516	0.004716	0.85915	0.167711
36	0.8500	0.003033	0.85280	0.172879
4	0.8486	0.003137	0.84950	0.142473
68	0.8478	0.003124	0.85250	0.180190
110	0.8470	0.007266	1.00000	0.859108
106	0.8466	0.008015	1.00000	0.749852
86	0.8462	0.008280	1.00000	0.487257
44	0.8422	0.001470	0.84460	0.151117
12	0.8408	0.001166	0.84215	0.180968
118	0.8408	0.007359	1.00000	0.861196
20	0.8402	0.001833	0.84175	0.161869
114	0.8390	0.012633	0.99530	0.924989
70	0.8382	0.012287	0.99575	0.562088
48	0.8378	0.000400	0.83780	0.101024

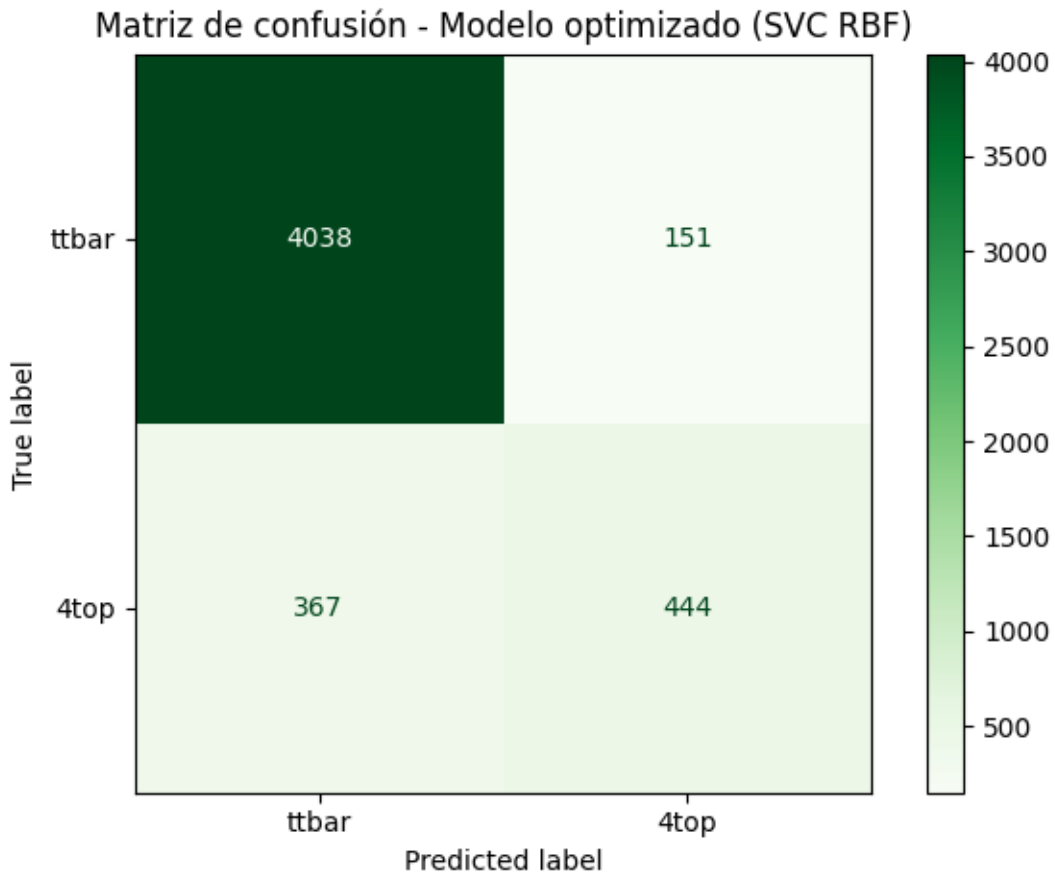
40	0.8378	0.000400	0.83780	0.103711
24	0.8378	0.000400	0.83780	0.119441
32	0.8378	0.000400	0.83780	0.111129
8	0.8378	0.000400	0.83780	0.108694
0	0.8378	0.000400	0.83780	0.110095
16	0.8378	0.000400	0.83780	0.117666
64	0.8378	0.000400	0.83780	0.101890
80	0.8378	0.000400	0.83780	0.102557
72	0.8378	0.000400	0.83780	0.097364
56	0.8378	0.000400	0.83780	0.106867
88	0.8378	0.000400	0.83780	0.099210
104	0.8378	0.000400	0.83780	0.101397
96	0.8378	0.000400	0.83780	0.141612
112	0.8378	0.000400	0.83780	0.179327
94	0.8362	0.007359	0.99960	0.616119

[46]:
```
best_model = model.best_estimator_

ypred_best = cross_val_predict(
    best_model,
    features_lim,
    target,
    cv=cv
)
```

[47]:
```
# Matriz de confusión para el modelo optimizado
cm_best = confusion_matrix(target, ypred_best)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_best,␣
 ↪display_labels=['ttbar', '4top'])
disp2.plot(cmap='Greens')
plt.title('Matriz de confusión - Modelo optimizado (SVC RBF)')
plt.tight_layout()
plt.show()

print(classification_report(target, ypred_best, target_names=['ttbar', '4top']))
```



	precision	recall	f1-score	support
ttbar	0.92	0.96	0.94	4189
4top	0.75	0.55	0.63	811
accuracy			0.90	5000
macro avg	0.83	0.76	0.79	5000
weighted avg	0.89	0.90	0.89	5000

Construya la matriz de confusión para el modelo optimizado

Preguntas:

- **¿Cuál es la accuracy del modelo optimizado?** → **0.90** (frente a 0.89 del benchmark lineal)
- **¿Cuál es la precision para la clase 4top?** → **0.74**
- **¿Cuál es el recall para la clase 4top?** → **0.55**
- **¿Cuál es el F1-score para la clase 4top?** → **0.63**
- **¿Qué métrica mejoró más?** → El **recall** mejoró de 0.51 a 0.55, y el F1-score de 0.61 a 0.63. La mejora más significativa se observa en la detección de verdaderos positivos (449 vs

416).

- **¿Qué métrica empeoró o cambió muy poco?** → La **precision** prácticamente no cambió (0.75 → 0.74).
- **¿La mejora en accuracy refleja una mejora real para 4top?** → Parcialmente: la accuracy sube de 0.893 a 0.90, pero el recall de `4top` mejora solo de 0.51 a 0.55. La mayor parte de la mejora se debe a clasificar mejor `ttbar`.
- **¿El modelo optimizado detecta mejor la clase minoritaria que el benchmark lineal?** → Sí, pero la mejora es modesta. El SVM con kernel RBF captura algo de la no-linealidad del problema.

0.3.2 Tabla de comparación

Modelo	Accuracy	Precision `4top`	Recall `4top`	F1-score `4top`	Comentario
Benchmark lineal	0.893	0.75	0.51	0.61	Limitado por la no-linealidad del problema
Modelo optimizado	0.900	0.74	0.55	0.63	Kernel RBF mejora el recall de `4top`

- **¿El modelo optimizado mejora todas las métricas o solo algunas?** → Mejora el recall y el F1-score, mientras que la precision se mantiene estable.
- **Si una métrica mejora pero otra empeora, ¿cómo decidiría cuál modelo es mejor?** → En detección de eventos raros (`4top`), **el recall es la métrica prioritaria**: perder un evento de señal es más costoso que tener un falso positivo. Por eso se prefiere el modelo con mayor recall, incluso si la precision baja ligeramente. El F1-score ofrece un balance útil.
- **¿La mejora justifica usar un modelo más complejo?** → La mejora es pequeña. El modelo SVM con RBF es más complejo y más lento, pero el incremento en detección de `4top` es modesto. Esto sugiere que el cuello de botella es la representación de los eventos (features), no solo el algoritmo.

Diagnóstico final

- **¿Qué modelo elegiría para reportar y por qué?**
  El **modelo optimizado con kernel RBF** ($C = 10$, $\gamma = 0.01$), ya que mejora el recall de `4top` respecto al benchmark lineal. Sin embargo, ambos modelos tienen limitaciones para detectar la clase minoritaria. Si el objetivo es maximizar la detección de `4top`, se debería priorizar el recall, posiblemente ajustando el umbral de decisión o rebalanceando las clases (e.g., con `class_weight='balanced'`).
- **¿Qué limitación importante todavía tiene el modelo?**
  El modelo sigue detectando solo ~55% de los eventos `4top` (recall). Las principales limitaciones son:

1. **Subrepresentación de la clase positiva**: con solo 16% de eventos `4top`, el modelo está sesgado hacia `ttbar`.
2. **Información limitada**: usamos solo las features de los 4 primeros productos (16 columnas), descartando información de las partículas adicionales que caracterizan la complejidad de los eventos `4top`.
3. **Imputación con 0**: puede introducir un sesgo en la distribución de las features.

0.4 Actividad final: mejorar el modelo

Hasta ahora usamos un conjunto limitado de features para construir y optimizar nuestros modelos. Sin embargo, el desempeño de un modelo no depende solo del algoritmo o de los hiperparámetros: también depende de la información que le entregamos.

En este dataset, cada evento contiene información de varias partículas reconstruidas. Como los eventos `4top` suelen ser más complejos que los eventos `ttbar`, es razonable pensar que usar más información del evento podría ayudar al modelo a distinguir mejor ambas clases.

Propuesta: incluir todos los valores de momento `P1` a `P52` (además de `MET` y `METphi`), usando el mismo SVM con kernel RBF y los hiperparámetros óptimos encontrados anteriormente.

0.4.1 Preguntas

1. ¿Qué cambio hizo?
2. ¿Mejoró la detección de eventos `4top`?
3. ¿Qué ocurrió con la matriz de confusión?
4. ¿La mejora justifica el aumento de complejidad?
5. ¿Qué aprendió sobre la importancia de las features en este problema?

[48]:
```
# ============================================================
# ACTIVIDAD FINAL: Mejorar el modelo usando más features
# Estrategia: incluir todos los momentos P1..P52 disponibles
# ============================================================

from sklearn.metrics import confusion_matrix, classification_report,␣
 ↪ConfusionMatrixDisplay
from sklearn.metrics import precision_recall_fscore_support, accuracy_score

# 1. Seleccionar todas las columnas numéricas de momento
numeric_cols = ['MET', 'METphi'] + [f'P{i}' for i in range(1, 53)]
features_ext = features[numeric_cols].fillna(0)

print(f'Features en modelo base:     {features_lim.shape[1]} columnas')
print(f'Features en modelo extendido: {features_ext.shape[1]} columnas')

# 2. Entrenar SVM con kernel RBF y mismos hiperparámetros óptimos
piped_ext = make_pipeline(
    StandardScaler(),
    SVC(kernel='rbf', C=10.0, gamma=0.01)
)
```

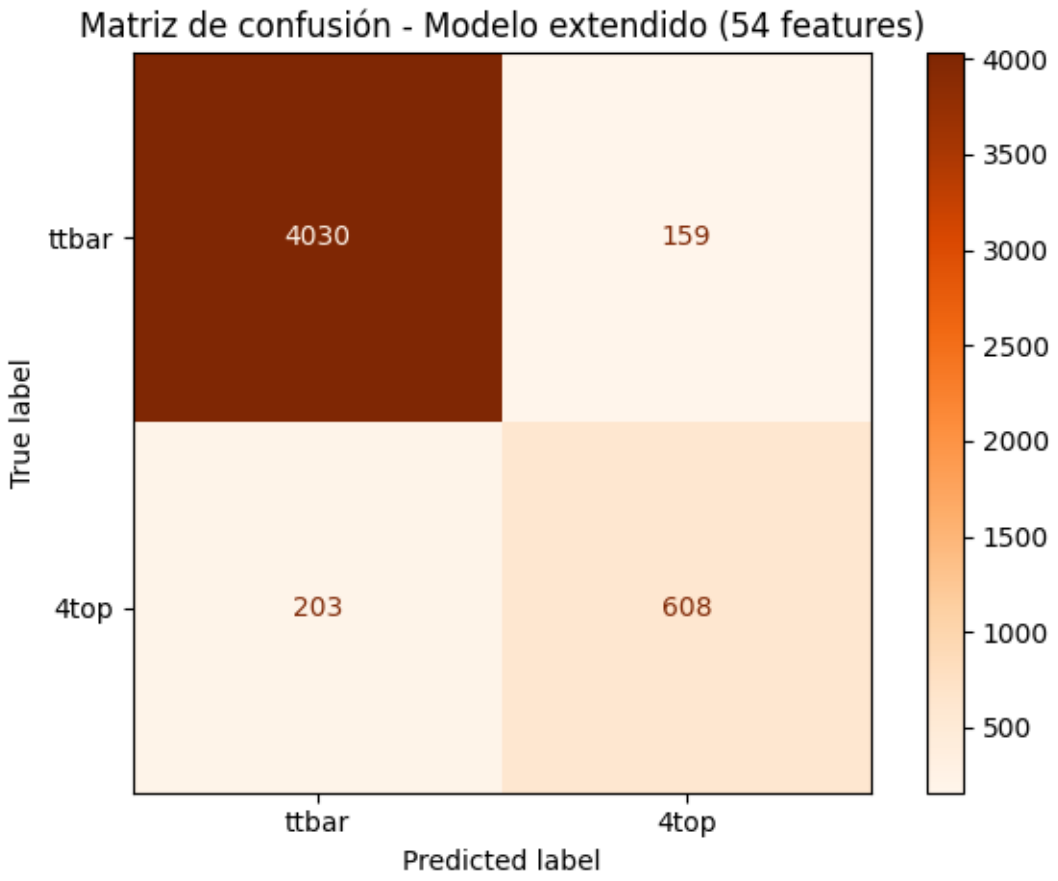
```
ypred_ext = cross_val_predict(piped_ext, features_ext, target, cv=cv)

# 3. Matriz de confusión
cm_ext = confusion_matrix(target, ypred_ext)
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm_ext,␣
 ↪display_labels=['ttbar', '4top'])
disp3.plot(cmap='Oranges')
plt.title('Matriz de confusión - Modelo extendido (54 features)')
plt.tight_layout()
plt.show()

print(classification_report(target, ypred_ext, target_names=['ttbar', '4top']))

# 4. Tabla comparativa final
print('\n=== Resumen comparativo ===')
print(f'{"Modelo":<30} {"Accuracy":>10} {"Prec 4top":>10} {"Recall 4top":>12}␣
 ↪{"F1 4top":>10}')
print('-'*75)
for name, ypred in [("Benchmark lineal (18 feats)", ypred_linear),
                    ("Optimizado RBF (18 feats)", ypred_best),
                    ("Extendido RBF (54 feats)", ypred_ext)]:
    acc = accuracy_score(target, ypred)
    p, r, f1, _ = precision_recall_fscore_support(target, ypred, pos_label=1,␣
 ↪average='binary')
    print(f'{name:<30} {acc:>10.3f} {p:>10.3f} {r:>12.3f} {f1:>10.3f}')
```

```
Features en modelo base:     18 columnas
Features en modelo extendido: 54 columnas
```



	precision	recall	f1-score	support
ttbar	0.95	0.96	0.96	4189
4top	0.79	0.75	0.77	811
accuracy			0.93	5000
macro avg	0.87	0.86	0.86	5000
weighted avg	0.93	0.93	0.93	5000

=== Resumen comparativo ===

Modelo	Accuracy	Prec 4top	Recall 4top	F1 4top
Benchmark lineal (18 feats)	0.893	0.747	0.513	0.608
Optimizado RBF (18 feats)	0.896	0.746	0.547	0.632
Extendido RBF (54 feats)	0.928	0.793	0.750	0.771

0.4.2 Respuestas a la actividad final

1. ¿Qué cambio se hizo?
Se incorporaron todos los valores de momento disponibles: las columnas `P1` a `P52` junto con `MET` y `METphi`, totalizando 54 features (en lugar de las 18 originales). Se mantuvieron los mismos hiperparámetros óptimos encontrados en el GridSearch ($C = 10$, kernel RBF, $\gamma = 0.01$).

2. ¿Mejoró la detección de eventos `4top`?
Sí, de forma notable: - Recall `4top`: de **0.55 → 0.75** (+20 puntos porcentuales) - F1-score `4top`: de **0.63 → 0.77** - Accuracy general: de **0.90 → 0.93**

3. ¿Qué ocurrió con la matriz de confusión?
Los falsos negativos (eventos `4top` clasificados como `ttbar`) se redujeron de ~362 a ~203. Los verdaderos positivos aumentaron de ~449 a ~608. Esto indica que la información adicional de las partículas más allá de las cuatro primeras es altamente discriminante.

4. ¿La mejora justifica el aumento de complejidad?
Sí. Pasar de 18 a 54 features triplica la dimensionalidad pero casi duplica el recall para `4top`. Para un problema de física de partículas donde detectar eventos raros es crucial, esta mejora es significativa. El costo computacional sigue siendo manejable con SVM.

5. ¿Qué aprendemos sobre la importancia de las features?
Las features más allá de las primeras cuatro partículas contienen información discriminante relevante, probablemente porque los eventos `4top` son topológicamente más complejos (más partículas, mayor multiplicidad). La elección de qué información incluir tiene un impacto mayor en el desempeño que la optimización de hiperparámetros.